

TEMA 0 – Introducción

Objetivo del tema

En este tema aprenderás los conceptos básicos del Sistema Operativo Android, sus características, componentes, sus versiones y niveles de API.

0.1 Un poco de historia

Los primeros dispositivos móviles sólo se podían programar en **bajo nivel**. Esto obligaba a los programadores a conocer íntimamente el hardware sobre el que trabajaban. Poco a poco la sofisticación de los dispositivos aumentó hasta que los programadores pudieron abstraerse por completo del hardware. De esta época data el **Symbian**, un sistema operativo para móviles que contaba con la posibilidad de instalar aplicaciones. Aun así, este tipo de plataformas requería utilizar código C/C++ razonablemente complejo que hacía uso de librerías de bajo nivel.

Otro problema era que tampoco había un único sistema operativo, por lo que era casi imposible estar al tanto de todos ellos, de los diferentes tipos de hardware, etc. La solución fue dotar a los sistemas operativos de dar soporte a **Java ME**. Al tratarse de un lenguaje multiplataforma, Java hacía posible desarrollar aplicaciones abstrayéndose del hardware, aunque a costa de estar limitado por la propia máquina virtual de Java.

Esto motivó la aparición de Android, cuya primera versión data del 2009 coincidiendo con la proliferación de los *smartphones* y las pantallas táctiles. Desde entonces han ido apareciendo diferentes versiones, desde la **1.5 Cupcake** (basada en el núcleo de Linux 2.6.27) hasta las actuales, orientadas a tablets, televisiones, etc. Cada versión del Sistema Operativo tenía un nombre inspirado en la repostería y que seguía un orden alfabético, al menos hasta a versión **1.9 Pie**. A partir de entonces las versiones pasan a llamarse Android 10, Android 11, etc.

Versión	Nombre	Versión	Nombre
1.0	A – Apple Pie	4.1/4.2/4.3	J – Jelly Bean
1.1	B – Banana Bread	4.4	K – Kit Kat
1.5	C – Cupcake	5.0/5.1	L – Lollipop
1.6	D – Donut	6.0/6.1	M – Marshmallow
2.0/2.1	E – Éclair	7.0/7.1.2	N – Nougat
2.2	F – Froyo	8.0	O – Oreo
2.3	G – Gingerbread	9.0	P – Pie
3.0/3.1/3.2	H – Honeycomb	10.0	Android 10
4.0	I – Cream Sandwich	11.0	Android 11

La última versión del sistema operativo (a junio de 2025) es la **Android 16**.

0.2 Características de Android

Android es un Sistema Operativo de **código abierto** para dispositivos móviles. Se programa principalmente en **Java** o en **Kotlin**, y su núcleo está basado en **Linux**. Tanto el S.O. como la plataforma de desarrollo están liberados bajo la licencia de **Apache**. Esta licencia permite a los desarrolladores añadir sus propias extensiones propietarias sin tener que ponerlas en manos de la comunidad de *software libre*. A demás, el hecho de ser *open source* le otorga varias ventajas:

- Una gran comunidad de desarrollo, con APIs completas y documentación libre.
- Se puede desarrollar en cualquier plataforma: Linux, Mac, Windows...
- Cualquier fabricante puede desarrollar un dispositivo que funcione con Android.
- Se ahorra en el desarrollo de un S.O., API para gráficos, acceso al hardware, etc.

0.3 Componentes de Android

El Sistema Operativo Android está formado por los siguientes componentes:

- **Núcleo** basado en Linux para el manejo de memoria, procesos y hardware.
- **Bibliotecas open source** para el desarrollo de aplicaciones: SQLite, Webkit, OpenGL, etc.
- **Entorno de ejecución** para las aplicaciones Android.
- Un **framework** de desarrollo que pone a disposición de las aplicaciones los servicios del móvil, como geolocalización, sensores, proveedores de contenido, etc.
- Un **SDK** (kit de desarrollo) que incluye el Android Studio, emuladores, ejemplos, ...
- **Interfaz** adaptado a las pantallas táctiles.
- **Apps preinstaladas** que hacen al S.O. útil para el usuario desde el primer momento.
- **Google Play**, que permite a los desarrolladores publicar sus propias Apps.

0.4 Versiones de la API

Las diferentes versiones de Android tienen asociado un código numérico llamado **nivel de API**. Las versiones más modernas de Android tienen un nivel de API superior. Un nivel superior de API significa que tiene más funciones, más clases, etc. Por tanto, es capaz de hacer más cosas que un nivel de API inferior.

El concepto es similar al de los videojuegos. Si no tienes tanta memoria RAM, o una tarjeta gráfica potente, hay juegos que no te van a funcionar en tu viejo ordenador.

Cuando tú desarrollas en cierto nivel de API, usarás cosas que otro nivel inferior de API no tiene. Por tanto, tu APP no funcionará en versiones antiguas de Android. Si desarrollamos una App para un nivel de API de 29 (Android 10) no funcionará en un móvil que tenga instalado la versión Kit Kat porque esa versión de Android usa API de 24.

Nombre	Versión	Nivel API
Cupcake	1.5	3
Froyo	2.2.X	8
Kit Kat	4.4	19
Lollipop	5.0	21
Android 10	10.0	29

La última versión del sistema operativo (a junio de 2025) es la **Android 16** tiene un nivel de API 36.

Cuando desarrollemos una App habrá que indicarle tres cosas:

- **minSdkVersion:** El Android mínimo que necesita tu App para funcionar.
- **targetSdkVersion:** Para qué versión de Android has preparado tu App.
- **compileSdkVersion:** Con qué versión de Android compilas la App.

Importante – ¿Qué nivel de API usar?

Durante tu aprendizaje básico, no importará el nivel de API que escojas para tus Apps. Lo que vas a codificar es sencillo, y funcionará en casi cualquier API. Si no sabes cuál escoger, escoge API 29 (Android 10). O escoge la de tu móvil, si vas a desplegar en él tus Apps. Más adelante te diremos cómo escoger el nivel de API en tu proyecto y podrás ver el % estimado de móviles en el mundo en el que podrás funcionar.

TEMA 1 – Tu Primera App

🎯 Objetivo del tema

Vamos a realizar una primera App simple en Android Studio para que te vayas familiarizando con el IDE. No te preocupes si utilizamos un lenguaje de programación que no conoces (Kotlin), o de si desconoces la estructura de un proyecto Android. Simplemente queremos programar nuestra primera App *¡Hola, Mundo!*

1.1 ¿Qué es el Android Studio?

Es el entorno de desarrollo (**IDE**) oficial de Android, que se utiliza para crear aplicaciones móviles para Android. Permite elaborar código para tus Apps; diseñar las pantallas; compilar, ejecutar y depurar programas; y *emularlos* en una máquina virtual que imita a un dispositivo móvil de tu elección.

El **Android Studio** es un programa, complejo, potente y pesado. Consume muchos recursos del equipo en el que se ejecuta, tanto de RAM como de procesador, disco, etc. Como ejemplo, la emulación de un móvil sencillo requiere 10+ Gigas de disco duro. No es buena idea tener nunca dos Android Studio funcionando a la vez.

Si no lo has hecho ya, **instala** el Android Studio en tu equipo. Puedes descargarlo de internet. Si te pide que lo actualices, es conveniente que lo hagas antes de empezar a trabajar. Tradicionalmente, Google tiene la costumbre de sacar una gran actualización anual. Es conveniente tenerlo siempre actualizado.

La última versión del Android Studio (a octubre de 2025) es la **Otter 2**.

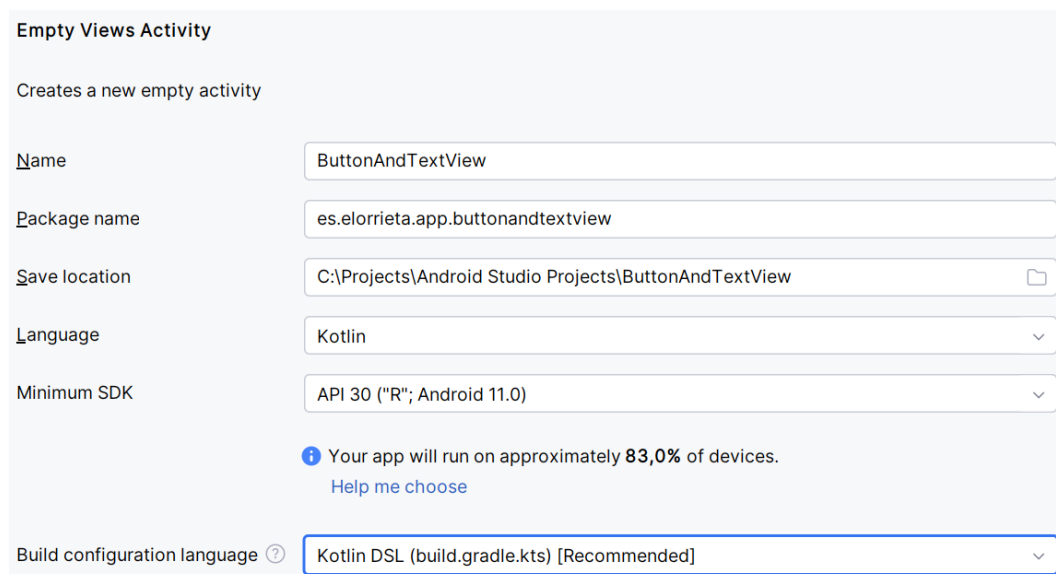
Android Studio permite cualquier versión del sistema operativo Android y cualquier nivel de API.

1.2 ¡Hola, mundo!

Para aprender el entorno de desarrollo Android Studio, vamos a realizar una App sencilla estilo *¡Hola Mundo!* Esta práctica la tienes solucionada: **P0 - ButtonAndTextViewKotlin.zip**

Paso 1 – Enunciado: Elabora una App que disponga de un botón un campo de texto El botón tiene que estar etiquetado como “Ok”. El campo de texto mostrará el texto “Pulsa el botón”. Cuando el usuario presiona el botón, el campo de texto cambiará a “Hola Mundo”. Si el usuario presione el botón de nuevo, el TextView mostrará otra vez “Pulsa el botón”.

Paso 2 –Nuevo Proyecto: Abre el Android Studio y crea un nuevo proyecto con **Create New Project**. En la ventana de **Project Template**, escogemos la plantilla **Empty View Activity**. Veremos entonces una ventana similar a esta donde definimos la Actividad principal de la App.



Empty Views Activity

Creates a new empty activity

Name: ButtonAndTextView

Package name: es.elorrieta.app.buttonandtextview

Save location: C:\Projects\Android Studio Projects\ButtonAndTextView

Language: Kotlin

Minimum SDK: API 30 ("R"; Android 11.0)

Information: Your app will run on approximately 83,0% of devices. [Help me choose](#)

Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

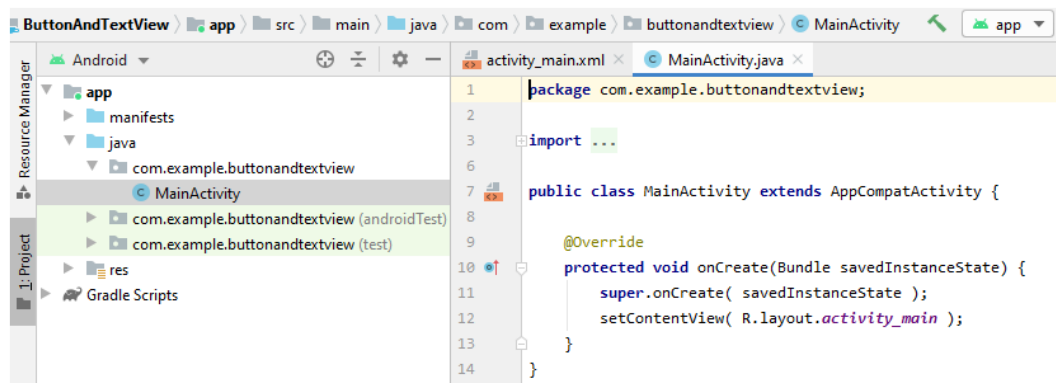
Completamos los campos de la siguiente forma:

- Nombre del proyecto **ButtonAndTextView**. El nombre del paquete se autocompletará automáticamente. Este es el paquete en el que se colocarán las clases del proyecto.
- En lenguaje, escoge Kotlin.
- Para la SDK, escoge el nivel de API 30.
- Para el lenguaje de configuración, deja el recomendado. Aquí se indica el script de compilación de la App, que trataremos más adelante.
- Dale a aceptar.

Importante – % de dispositivos

Android Studio nos muestra el % de móviles en todo el mundo en los que funcionará nuestra App. Este % cambia en base al nivel de API que escojamos. A cuanto mayor nivel de API, menos % habrá. Eso es así porque las App más modernas (nivel de API mayor) no funcionarán en un dispositivo antiguo (menor nivel de API).

Paso 3 –Interfaz: Una vez inicializado el proyecto, veremos una ventana similar a esta.



Fíjate en que:

- A la izquierda verás todas las carpetas y ficheros que conforman tu App. Todas las App tienen una estructura y organización similar.
- Se nos abren por defecto dos ficheros: **MainActivity.java** y **activity_main.xml**. Estos dos ficheros juntos describen una **actividad** de nuestra App.
 - ✓ **activity_main.xml** define cómo se ve la App en nuestro dispositivo.
 - ✓ **MainActivity.kt** cómo se comportan los componentes de esa parte visual.

Usaremos el **activity_main.xml** para ‘colocar’ el botón y el campo de texto, y en el **MainActivity.java** diremos ‘qué pasa’ cuando pulsamos el botón (eventos).

Vamos primero al **activity_main.xml**.

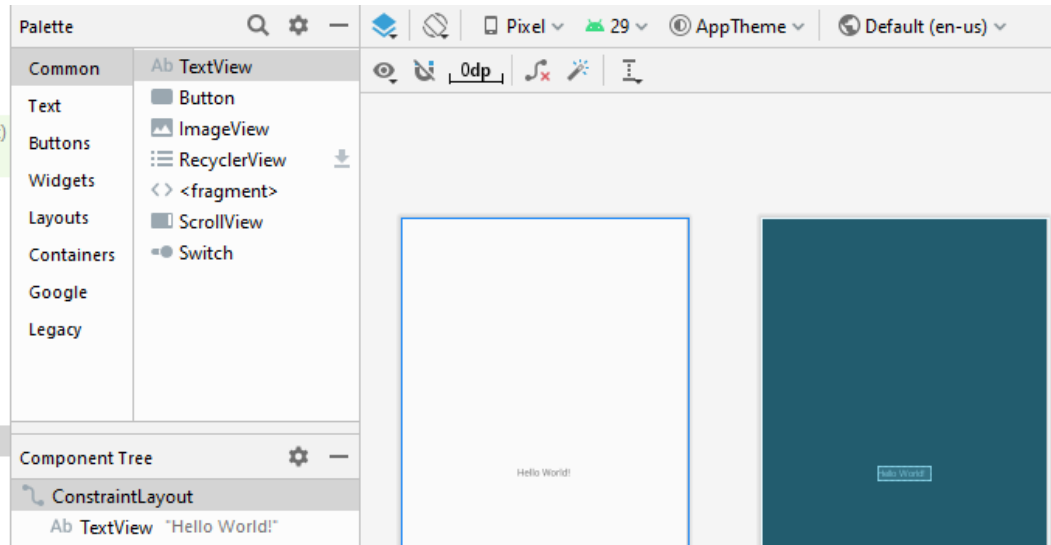
Este fichero se encuentra en **app/res/layout/activity_main.xml**. En la carpeta **/res** es donde se organizan todos los recursos de la App: Imágenes, sonidos, textos... cada uno en su subcarpeta.

Un ejemplo de para qué sirve cada fichero:

- **Strings.xml**: Referencias a textos de la App.
- **Styles.xml**: Definen los estilos de la vista, como un CSS.

Vamos con la ‘ventana’. Abre **activity_main.xml**. Aquí es donde añadirás el botón y el campo de texto a la ventana. Puedes hacerlo de dos maneras: con la herramienta visual de Android Studio, o editando el fichero a mano. El truco es usar la herramienta visual y retocar el xml a mano. Puedes cambiar de ambos modos a través del propio interfaz del Android Studio.

La herramienta visual se ve así:



Por defecto, al crear una nueva actividad se añade un **ConstraintLayout** y un **TextView** (campo de texto) con el texto “Hello World”.

Si editas el fichero a mano, se verá así:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/a
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   xmlns:tools="http://schemas.android.com/tools"
5   android:layout_width="match_parent"
6   android:layout_height="match_parent"
7   tools:context=".MainActivity">
8
9   <TextView
10     android:layout_width="wrap_content"
11     android:layout_height="wrap_content"
12     android:text="Hello World!"
13     app:layout_constraintBottom_toBottomOf="parent"
14     app:layout_constraintLeft_toLeftOf="parent"
15     app:layout_constraintRight_toRightOf="parent"
16     app:layout_constraintTop_toTopOf="parent" />
17
18 </androidx.constraintlayout.widget.ConstraintLayout>
```

Utilizando la herramienta visual, añade un nuevo **Button** (botón) a la ventana. El botón tiene que quedarte dentro del **ConstraintLayout**. Este layout te permite centrar los componentes mediante unos ‘muelles’ que se enganchan en las ‘bolas’ que tienen, y que sirve para centrarlos, mejor. Deberías enganchar cada componente con cada borde de la pantalla. Más adelante explicaremos qué es eso.

Abre el fichero **strings.xml**. Aquí añade los textos que se mostrarán en la App. En Android Studio los literales y textos nunca se ponen directamente, se extraen de este fichero de recursos. Tiene que quedar así:

```
<resources>
    <string name="app_name">ButtonAndTextView</string>
    <string name="text_ok">OK</string>
    <string name="text_pulsa">Pulsa el boton</string>
    <string name="text_hola_mundo">Hola Mundo</string>
</resources>
```

Esta forma de trabajar permite hacer cosas como cambiar el idioma de la App. Vuelve ahora al **activity_main.xml** y haz que el texto del **Button** se saque del strings.xml en lugar de ser un literal. Hay varias formas de hacerlo.

Una forma es mediante la barra de atributos que hay a la derecha. Esos atributos son todos los atributos existentes para un componente de la ventana, y son los mismos que el puedes poner en el xml. Obviamente, en el xml sólo aparecerán aquellos a los que hayas dado valor.

Asigna estos valores:

Para el **Button**:

- ID: button
- Text: @string/text_ok

Para el **TextView**:

- ID: textView
- Text: @string/text_pulsa

Si lo has hecho bien, los textos que aparecen ahora en la herramienta visual son los mismos que en el fichero de **strings.xml**.

Por el momento, ya hemos acabado con la **parte visual** de la App.

Un último consejo: ¿ves los warnings? Nunca dejes un warning sin solucionar... sobre todo teniendo al Android Studio detrás que, con un par de clicks, te auto soluciona la mayoría de los problemas.

Paso 4 –Lógica del negocio: Una vez definida la interfaz, vamos a darle lógica a la App. Abre la clase **MainActivity.kt**. Aquí se describe el comportamiento de cada uno de los elementos del interfaz. Por defecto, el Android Studio nos da este código:

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

El código de **Kotlin** es similar al de **Java**. Pero de Kotlin ya hablaremos en otro apartado.

Una de las peculiaridades de la arquitectura de Android es que no hay un `main ()`. En lugar de eso, la función **onCreate** se lanza automáticamente cuando se inicia la App. Date cuenta de que **onCreate** NO es un constructor. Aquí no se usan los constructores. De hecho, si intentas usar variables de clase, no se te van a inicializar y serán siempre nulas, pongas lo que pongas.

Más adelante explicaremos de esto cuando veamos el ciclo de vida de una actividad.

Las dos líneas de código que te escribe Android Studio por defecto (o las que sean) no se tocan. Nosotros escribiremos nuestro código debajo. Si te fijas, lo que hacen es vincular la **MainActivity.kt** (la lógica) con el **activity_main.xml** (la Vista).

Para poder utilizar el **Button** y el **TextView** (añadirles un comportamiento), es necesario “referenciarlos” de alguna forma. Y lo hacemos mediante una variable.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val textView : TextView = findViewById(R.id.textView)  
    val button : Button = findViewById(R.id.button)  
}
```

En Kotlin, las variables se definen así. Primero, se define la variable con **val** o **var**. Un **val** es una variable de solo lectura, cuyo valor no va a cambiar nunca. Equivale a poner final en Java. En cambio, si ponemos **var** estamos diciendo que podemos reasignar el valor de dicha variable. Por ejemplo, haciendo que button apunte a otro button. Los **dos puntos** sirven para decir el tipo de variable que es, en este caso, un TextView y un Button.

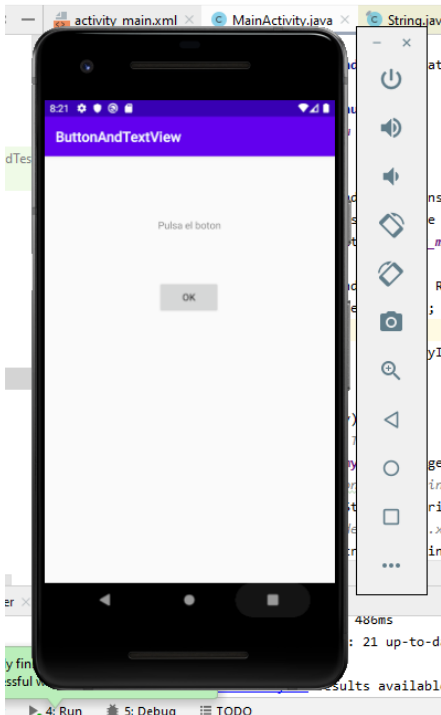
Indicamos ahora el evento al **Button** que hará cambiar el texto del **TextView**. Este bloque de código tiene que colocarse inmediatamente después del anterior y dentro del método **onCreate ()**.

```
button.setOnClickListener {  
  
    val textViewText = textView.text.toString()  
  
    val textOk = getString(R.string.text_click)  
    val textHello = getString(R.string.text_hello_world)  
  
    textView.text = if (textViewText == textOk) textHello else textOk  
}
```

Comentarios sobre el código:

- En la primera línea, cargamos en **textViewText** el contenido de **textView**. En Kotlin no se ponen los getters y setters, dado que todos los atributos de clase son por defecto públicos. Hacer `textView.text` equivale a hacer `textView.getText()` en Java.
- Cargamos en **textOk** y **textHello** los literales del fichero **strings.xml**.
- El if-else que ves es la forma resumida que tiene Kotlin de hacer una comparación y una asignación en una sola línea de código. No te despistes por cómo compara dos Strings iguales Kotlin; funciona como el equals de Java.

Paso 5 –Emulación: Vamos a **emular** un dispositivo Android. Si no tienes ningún dispositivo ya definido en tu Android Studio, puedes bajarte uno con la opción **Device Manager**. Yo tengo por defecto el **Píxel 2 API 29**, pero puedes tener tantos como quieras. Para hacerlo, sólo tienes que ir a la derecha del todo de tu interfaz de Android, pulsar **Device Manager** y darle al + (Add new Device).



Una vez bajado y seleccionado, bastaría con darle al botón de Run para que se ejecutase en el emulador. Se ejecutará entonces una Máquina Virtual que emulará el dispositivo móvil que has seleccionado. Ese 'móvil' funciona como un móvil real, incluso puedes conectarte a internet con él y sacar fotos.

Dado que la **emulación** es un proceso muy costoso en recursos para un equipo, notarás que todo va lento. Si tienes un equipo antiguo, empiezas a darle a los botones a lo loco, o intentas arrancar dos Android Studio a la vez... el equipo irá cada vez peor.

A veces puede que no despliegue bien la App en el emulador y tengas que borrar y reiniciar el emulador. No seas burro.

Afortunadamente, el Android Studio incorpora una herramienta de Debug que funciona muy bien. Es capaz de trazar tu App en caliente.

Y es más lenta que un caracol también...

Paso 5 –Instalando la App en tu móvil (Opcional): Si deseas instalar la App en tu dispositivo Android personal, deberás tener un cable USB y esperar a que el Android Studio te lo detecte. Debería de ser algo automático. Será necesario que antes habilites tu dispositivo en modo desarrollador.

Cada modelo lo hace de una manera diferente, deberías de mirarlo en Internet. En mi caso, tengo que acceder a **Ajustes → Sistema → Acerca del teléfono**, y bajar hasta la opción de **Número de compilación**. Una vez allí, **pulso siete veces seguidas** sobre esta opción. Si ya lo has hecho una vez, no hay que volver a hacerlo más adelante.

A continuación, tengo que conectar el cable y habilitar la depuración: **Ajustes → Sistema → Opciones del Desarrollador → Habilitar Depuración USB**. Si todo ha ido bien, el Android Studio detectará mi móvil y me aparecerá en la pestaña del **emulador**.

Cuando termines, deshabilita el modo Depuración, o tu móvil no funcionará correctamente. Pueden ocurrir cosas imprevistas, como pérdida de rendimiento o que se agote la batería rápidamente. Estás avisado.

No es obligatorio que uses tu móvil en clase.

Importante – ¡Paciencia!

Android Studio consume muchísimos recursos de tu equipo. Esto significa que a veces va lento, sobre todo al arrancar un proyecto nuevo o al importarlo. Y otras veces, se te pone a descargar cosas que no sabes muy bien qué son, o el emulador parece que no responde.

La mejor forma de trabajar con Android Studio, insisto, es desarrollar unas buenas prácticas. Déjale trabajar. Borra de vez en cuando la App del emulador, o resetéalo por completo. Y por favor, no seas vago y abras dos instancias del Android Studio para ir copiando y pegando código. Tardarás más que usando el Notepad y el explorador de archivos de Windows.

Ejercicio 1

Desarrolla una App en Android que disponga de un **Button**, un **TextView** y un **EditText**. Al arrancar la App, el TextView mostrará el texto “Introduce un número primo”, y el Button estará etiquetado como “Comprobar”. Por su parte, el EditText mostrará el texto por defecto “Tu número”.

Cuando se presione el Button, se comprobará si su texto es “Comprobar” o “Reiniciar”.

- Si es “Comprobar”, se extraerá el número introducido en el EditText, y se comprobará si es un número primo. Si no es primo, el TextView mostrará el texto “No es un número primo” y el Button se etiquetará como “Reiniciar”. Si es primo, el TextView mostrará el texto “Es un número primo” y el Button se etiquetará como “Reiniciar”.
- Si es “Reiniciar”, el TextView mostrará el texto “Introduce un número primo” y el Button se etiquetará como “Comprobar”.
- Si se produce una excepción, por el motivo que sea, el programa NO debe determinar. En lugar de eso, el TextView mostrará el texto “No es un número primo” y el Button se etiquetará como “Reiniciar”.

Investiga la manera de conseguir que el EditText sólo permita la entrada de números de hasta 4 cifras. Para comprobar si un número primo, implementa una clase Operations en un paquete operations (no junto a la actividad). En ella define una función isPrimeNumber (number : int), que retornará True si el número es primo, y False en cualquier otro caso.

Usa el strings.xml para los textos.